Module 05 | From Activation to Loss Function

My Solution to Practice Problems

Table of Contents

Q01. Why do we need an activation function in a neural network? What happens if we don't use one?

We need activation functions in the neurons of a neural network to introduce non-linearity in a way that coordinates the linear weighted sums of the neurons to collectively approximate complex non-linear relationship.

If we use activation function in no neuron in a neural network, the whole network is no more than linear in its capability. Because of that limited capability, the network is replacable by any single linear neuron. The depth of the network is useless, as it does not introduce any expressive power that is already not in its unit neurons.

Q02. Explain why sigmoid is called a "squashing function".

The sigmoid function, given the entire set of real numbers, namely the interval $(-\infty, +\infty)$, as input, turns every input, whatever the sign is and however large the absolute value is, into a number in the

tiny interval $(0, 1)$. This behavior can be expressed by saying that it compresses even large numbers of either sign into small numbers. That's why it is called a `Squashing Function`.

Q03. Why is ReLU more popular than Sigmoid in deep neural networks?

- ReLU is computationally more effient.
- ReLU relieves us from `Vanishing Gradient` Problem of Sigmoid Function.
- ReLU allows sparse activation, leading to a network that is lighter, more parameter-efficient, and less prone to overfitting compared to Sigmoid's dense, non-zero outputs.

Q04. What problem does ReLU solve compared to Sigmoid?

- ReLU is computationally more effient.
- ReLU relieves us from `Vanishing Gradient` Problem of Sigmoid Function.
- ReLU allows sparse activation, leading to a network that is lighter, more parameter-efficient, and less prone to overfitting compared to Sigmoid's dense, non-zero outputs.

Q05. When should we use Softmax instead of Sigmoid?

We should use Softmax instead of Sigmoid when our model needs to perform multi-class classification where the target classes are mutually exclusive (each input belongs to exactly one category).

Q06. Why is Binary Cross Entropy preferred over MSE for classification?

$$\text{MSE Loss} = (y - \hat{y})^2$$

$$\text{Binary Cross Entropy Loss} = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

We know, `MSE Loss` is used in Linear Regression. I will subject my perspective to this.

During training,

- in Linear Regression, we try to find a line that `fits` the data points.
- in Binary Classification, we try to find a line that `separates` the data points.

In Linear Regression, a data point that is not on the line has error associated with the model's prediction on it, no matter which side of the line the point lies, captured by `squaring` in MSE. Only, the farther it is from the line, the larger is the cost induced by the error, which is too captured by MSE.

In Binary Classification, the expection from Loss Function is different.

- It does not matter, unlike in Linear Regression, how far a data point is from the line, when the prediction is correct, the error must be 0— this requirement is met by `MSE` for deterministic prediction, but not for probabilistic prediction.
- In Linear Regression, the farther a data point is from the line, the more we **punish** the model. On contraru, in Binary Classification, the farther a data point with correct prediction is from the line, the more we **reward** it for confidence in prediction. The sigmoid function expresses this confidence probabilistically.
- In Linear Regression, we do not care which side of the line a data point lies. But in Binary Classification, we do— both `which side` and `how far`. Two sides is captured by two terms in `BCE Loss`, one term of each side. And with probabilistic prediction, `how far` is captured by $\log$ function and the minus sign.

So, among `MSE` and `Binary Cross-Entropy`, for Binary Classification, `Binary Cross-Entropy` is not `preferred`, but the ***only right choice***— and this right choice demands `Sigmoid Function` with it.

Q07. If true label $y = 1$ and predicted probability $\hat{y} = 0.9$, calculate the Binary Cross Entropy loss.

$$\begin{aligned}\text{BCE} &= -\left[y \log(\hat{y}) + (1-y)\log(1-\hat{y})\right] \\ &= -[1 \times \log(0.9) + 0] \\ &\approx 0.05\end{aligned}$$

Q08. If $y = 0$ and $\hat{y} = 0.8$, calculate the loss.

$$\begin{aligned}\text{BCE} &= -\left[y \log(\hat{y}) + (1-y)\log(1-\hat{y})\right] \\ &= -[0 + (1-0) \times \log(0.2)] \\ &\approx 0.70\end{aligned}$$

Comparison between Q07 and Q08

A confident correct prediction, as in **Q07**, incurs less cost than a confident wrong prediction, as in **Q08**.

Q09. Compute $\sigma(0)$, $\sigma(2)$, and $\sigma(-2)$.

$$\sigma(x) \equiv \frac{1}{1+e^{-x}}$$

$$\sigma(0) = \frac{1}{1+e^{-0}} = \frac{1}{2} = 0.5$$

$$\sigma(2) = \frac{1}{1+e^{-2}} \approx 0.88$$

$$\sigma(-2) = \frac{1}{1+e^{2}} \approx 0.12$$